Δομές Δεδομένων

3η Εργασία

Η εκτέλεση του προγράμματος, γίνεται από την γραμμή εντολών με την μορφή:

Java RandomizedBST <input_file>

Αν δεν δοθεί το input_file, τότε φορτώνονται τα περιεχόμενα του αρχείου **input.txt** που υπάρχει μέσα στον φάκελο src.

Μέρος Α

Υλοποιήθηκε η κλάση LargeDepositor (αρχείο **LargeDepositor.java**) σύμφωνα με την εκφώνηση της εργασίας και να υπερφορτώθηκε κατάλληλα η μέθοδος toString για να μας χρησιμεύσει για την εκτύπωση αποτελεσμάτων, δημιουργήθηκαν επίσης setters και getters.

Η κλάση TreeNode υλοποιήθηκε μέσα στην κλάση RandomizedBST (αρχείο **RandomizedBST.java**).

Μέρος Β

Στο Μέρος Β, υλοποιήσαμε, τις παρακάτω μεθόδους σύμφωνα με τις οδηγίες της εργασίας:

- **void insert(LargeDepositor item)**: εισάγει στο δέντρο έναν νέο κόμβο που περιέχει τον μεγαλοκαταθέτη του αντικειμένου item. Εάν υπάρχει ήδη μεγαλοκαταθέτης με τον ίδιο ΑΦΜ στο δέντρο, τυπώνει μήνυμα λάθους και τερματίζει, χωρίς να κάνει καμία εισαγωγή. Η μέθοδος καλεί την **insertR(insertR(TreeNode h, LargeDepositor item)** η οποία τοποθετεί με πιθανότητα 1/(N+1) και με την βοήθεια της **insertAsRoot(item, h)** το item στην ρίζα του δέντρου (μέσω περιστροφών **rotR(h)**, **rotL(h)**), διαφορετικά κάνει αναδρομικά εισαγωγή στο κατάλληλο υποδέντρο.

  Η μέθοδος insert(LargeDepositor item) για την εισαγωγή ενός νέου κόμβου σε ένα τυχαιοποιημένο δυαδικό δέντρο αναζήτησης εκτελείται σε λογαριθμικό χρόνο σε σχέση με τον αριθμό των κόμβων στο δέντρο, στην περίπτωση που το δέντρο είναι κατά προσέγγιση ισορροπημένο. Σε μη ισορροπημένου Δέντρου, η πολυπλοκότητα μπορεί να φτάσει στο O(N) στη χειρότερη περίπτωση, όπου n είναι το πλήθος των κόμβων του δέντρου. Αυτό συμβαίνει όταν το δέντρο εκφυλίζεται σε μια γραμμική δομή (π.χ. κάθε νέος κόμβος εισάγεται πάντα στο δεξί ή αριστερό άκρο του δέντρου).

- **void load(String filename)**: αφορά τη φόρτωση δεδομένων από ένα αρχείο κειμένου και την εισαγωγή τους στο δυαδικό δέντρο αναζήτησης. Η μέθοδος αρχικά ανοίγει το αρχείο κειμένου με το δοθέν όνομα filename και διαβάζει κάθε γραμμή του αρχείου. Για κάθε γραμμή, η μέθοδος δημιουργεί ένα νέο αντικείμενο LargeDepositor με τα δεδομένα που

αναγνώστηκαν και χρησιμοποιώντας την **insert(LargeDepositor item)** το αντικείμενο εισάγεται στο δυαδικό δέντρο αναζήτησης.

- **void updateSavings(int AFM, double savings)**: χρησιμοποιεί την βοηθητική αναδρομική μέθοδο **searchNodeForUpdate(root, AFM, savings)** η οποία αναζητά τον κόμβο με τον συγκεκριμένο ΑΦΜ στο δέντρο. Αν ο κόμβος βρεθεί, ενημερώνεται η τιμή των αποταμιεύσεών του και επιστρέφεται ο κόμβος. Αν δεν βρεθεί κόμβος με τον συγκεκριμένο ΑΦΜ, επιστρέφεται null και εμφανίζεται ένα μήνυμα λάθους.
  Η πολυπλοκότητα της ενημέρωσης εξαρτάται κυρίως από την αναζήτηση του στοιχείου με τον συγκεκριμένο ΑΦΜ μέσα στο δέντρο. Σε ένα ιδανικά ισορροπημένο δυαδικό δέντρο αναζήτησης, η αναζήτηση λειτουργεί με λογαριθμική πολυπλοκότητα, δηλαδή O(log n), όπου n είναι το πλήθος των κόμβων στο δέντρο. Αφού βρεθεί ο κατάλληλος κόμβος, η ενημέρωση του πεδίου savings είναι μια λειτουργία σε σταθερό χρόνο, δηλαδή O(1). Συνδυάζοντας τα παραπάνω, η συνολική πολυπλοκότητα της updateSavings είναι κυρίως επηρεασμένη από την αναζήτηση και κατά προσέγγιση ισούται με O(log N) σε ένα ισορροπημένο δέντρο. Στην χειρότερη περίπτωση μπορεί να φτάσει στο O(N), όταν το δέντρο εκφυλίζεται σε μια γραμμική δομή.

- **LargeDepositor searchByAFM(int AFM**): ακολουθεί τον αλγόριθμο της δυαδικής αναζήτησης σε δυαδικό δέντρο αναζήτησης για να βρει τον καταθέτη με το δοσμένο ΑΦΜ. Δεδομένου ότι ο ΑΦΜ είναι μοναδικός για κάθε χρήστη, αυτός ο αλγόριθμος ακολουθεί τη διαδρομή αναζήτησης στο δέντρο βάσει του ΑΦΜ, επιστρέφοντας τον κόμβο όταν βρεθεί ο συγκεκριμένος ΑΦΜ ή null αν δεν βρεθεί. Καλεί την βοηθητική αναδρομικά συνάρτηση **searchNode(TreeNode node, int AFM)**, που αναζητά τον κόμβο με τον συγκεκριμένο ΑΦΜ στο δέντρο.
  Σε ένα ιδανικά ισορροπημένο δυαδικό δέντρο αναζήτησης, η πολυπλοκότητα είναι O(log N), όπου N είναι το πλήθος των κόμβων του δέντρου.

- **Lista searchByLastName(String last_name):** Ψάχνει στο δέντρο για την ύπαρξη μεγαλοκαταθετών με το συγκεκριμένο επίθετο. Επειδή ενδέχεται να υπάρχουν περισσότερα του ενός άτομα με το ίδιο επίθετο, η μέθοδος θα πρέπει να επιστρέφει μια λίστα μονής σύνδεσης με αντικείμενα τύπου LargeDepositor, αν δεν υπάρχει κάποιος με αυτό το επίθετο, απλά επιστρέφει nul.

  Δημιουργήσαμε μια δική μας απλά συνδεδεμένη λίστα (**Lista.java**) η οποία εκτός από τα δεδομένα item και next, περιέχει και τις μεθόδους **appendToLista(LargeDepositor newItem)** και **printLista()**. Η **appendToLista** προσθέτει ένα νέο κόμβο στο τέλος της λίστας, έτσι επιτρέπει την εύκολη επέκταση της λίστας με νέα στοιχεία. Η **printLista** τυπώνει τα στοιχεία όλων των κόμβων της λίστας.

  Η **searchByLastName(String last_name)** χρησιμοποιεί την βοηθητική μέθοδο **searchByLastNameRecursive(TreeNode node, String lastName, Lista list)**, διασχίζει το δέντρο αναδρομικά και όταν βρίσκει έναν καταθέτη με το δοθέν επίθετο, τον προσθέτει στη λίστα.

Σε ένα ιδανικά ισορροπημένο δυαδικό δέντρο αναζήτησης, η πολυπλοκότητα είναι O(log N), όπου N είναι το πλήθος των κόμβων του δέντρου. Στην χειρότερη περίπτωση μπορεί να φτάσει στο O(N), όταν το δέντρο εκφυλίζεται σε μια γραμμική δομή.

- **void remove(int AFM)**: Η μέθοδος αυτή αφαιρεί το αντικείμενο με τον συγκεκριμένο ΑΦΜ (αν υπάρχει στο δέντρο). Χρησιμοποιείται η **searchByAFM(AFM)**, που περιγράψαμε προηγουμένως, για να εντοπίσει τον κόμβο με το συγκεκριμένο ΑΦΜ. Όταν εντοπιστεί ο κόμβος, χρησιμοποιείται η μέθοδος **removeR(root, AFM)** όπως την έχουμε δει από τις διαφάνειες. Η **removeR()** αναζητά αναδρομικά και αφαιρεί τον κόμβο με τον δοθέντα ΑΦΜ. Αν ο κόμβος προς αφαίρεση βρεθεί, χρησιμοποιείται η **joinLR(TreeNode a, TreeNode b)** για να συνδυάσει τα δύο υποδένδρα που ήταν συνδεδεμένα με τον αφαιρεθέντα κόμβο. Η joinLR συνδυάζει δύο υποδένδρα, χρησιμοποιώντας την **partR(TreeNode h, int k)** για να φέρει το μικρότερο στοιχείο του δεξιού υποδένδρου στη ρίζα. Οι **rotR()** και **rotL()** υλοποιούν τις δεξιές και αριστερές περιστροφές αντίστοιχα. Η size επιστρέφει τον αριθμό των κόμβων σε ένα υποδέντρο.
  Όπως είδαμε η μέθοδος remove έχει δύο βασικές λειτουργίες: αναζήτηση του κόμβου προς αφαίρεση και επαναδιάταξη του δέντρου μετά την αφαίρεση του κόμβου. Η πολυπλοκότητα της μεθόδου εξαρτάται κυρίως από αυτές τις δύο λειτουργίες. Η αναζήτηση του κόμβου προς αφαίρεση ακολουθεί τη λογική της αναζήτησης σε δυαδικό δέντρο αναζήτησης, σε ένα ιδανικά ισορροπημένο θα έχει λογαριθμική πολυπλοκότητα, δηλαδή O(log N), όπου n είναι το πλήθος των κόμβων στο δέντρο. Σε μη ισορροπημένο δέντρο, αυτό μπορεί να φτάσει το O(N) στη χειρότερη περίπτωση. Η επαναδιάταξη του δέντρου εξαρτάται από τη δομή του δέντρου και τη θέση του κόμβου προς αφαίρεση. Στην περίπτωση της αφαίρεσης ενός κόμβου με δύο παιδιά, μπορεί να απαιτηθεί η εύρεση του άμεσου προκατόχου ή του άμεσου διαδόχου, η οποία είναι επίσης μια λογαριθμική λειτουργία σε ένα ισορροπημένο δέντρο. Συνδυάζοντας τα προηγούμενα, η συνολική πολυπλοκότητα της remove σε ένα ισορροπημένο δέντρο είναι συνήθως O(log N), ενώ σε ένα μη ισορροπημένο δέντρο μπορεί να φτάσει στο O(N) στη χειρότερη περίπτωση.

- **double getMeanSavings()**: Η μέθοδος αυτή υπολογίζει το μέσο ποσό καταθέσεων με βάση τους καταθέτες που είναι αποθηκευμένοι στο δέντρο. Χρησιμοποιείται η βοηθητική μέθοδος **sumSavings(root)**, για να βρεθεί το συνολικό ποσό και στη συνέχεια το διαιρούμε με τον αριθμό των κόμβων στο δέντρο (root.N) για να υπολογίσουμε το μέσο ποσό. Η **sumSavings(root)** είναι μια αναδρομική μέθοδος, που διασχίζει το δέντρο με την ενδοδιατεταγμένη (in-order αριστερό υποδέντρο – ρίζα – δεξί υποδέντρο) μέθοδο.
  Η πολυπλοκότητα αυτής της μεθόδου εξαρτάται από τον αριθμό των κόμβων στο δέντρο. Η διάσχιση του δέντρου γίνεται μέσω μιας αναδρομικής διαδικασίας, η οποία επισκέπτεται κάθε κόμβο ακριβώς μία φορά. Επομένως, η πολυπλοκότητα της διάσχισης είναι O(N), όπου N είναι ο συνολικός αριθμός των κόμβων στο δέντρο.

- **void printTopLargeDepositors(int k)**: Η μέθοδος αυτή βρίσκει και τυπώνει τα στοιχεία για τους k πιο ύποπτους για φοροδιαφυγή μεγαλοκαταθέτες σε φθίνουσα σειρά ως προς το

πόσο ύποπτοι είναι, σύμφωνα με τις προδιαγραφές που τέθηκαν από την εργασία, σχετικά με το πόσο ύποπτος για φοροδιαφυγή είναι κάποιος μεγαλοκαταθέτης.
Για την υλοποίηση χρησιμοποιούμε μια **μεγιστοστρεφή** ουρά προτεραιότητας PQ (αρχείο **PQ.java**) η οποία έχει μια εσωτερική κλάση HeapNode για την αποθήκευση των καταθετών και των αντίστοιχων σκορ και διαθέτει τις μεθόδους insert, swim, sink, swap, removeMax και printSuspects, στα πρότυπα της εργασίας 2.
Η printTopLargeDepositors(int k), χρησιμοποιεί την βοηθητική μέθοδο **fillPQ(root, maxHeap)** η οποία διασχίζει το δέντρο και εισάγει κάθε καταθέτη στην ουρά προτεραιότητας με το αντίστοιχο σκορ. Η μέθοδος printSuspects() στη συνέχεια εκτυπώνει τους k πιο ύποπτους καταθέτες από την ουρά προτεραιότητας.
Η πολυπλοκότητα αυτής της μεθόδου εξαρτάται από τρεις κύριες διαδικασίες:
Διάσχιση του Δέντρου (**fillPQ(root, maxHeap)**): Η μέθοδος διασχίζει όλο το δέντρο για να εξετάσει κάθε καταθέτη. Η διάσχιση ενός δέντρου έχει γραμμική πολυπλοκότητα σε σχέση με τον αριθμό των κόμβων στο δέντρο, δηλαδή **O(N)**.
Εισαγωγή στον Δυαδικό Σωρό (**maxHeap.insert(node.item, score)**: Για κάθε καταθέτη, γίνεται μια εισαγωγή στον δυαδικό σωρό. Η εισαγωγή σε ένα δυαδικό σωρό έχει πολυπλοκότητα **O(log k)**, όπου k είναι το πλήθος των στοιχείων στο σωρό. Ωστόσο, εφόσον ο σωρός περιορίζεται στα k στοιχεία, η πολυπλοκότητα των εισαγωγών παραμένει ελεγχόμενη και δεν εξαρτάται άμεσα από το N.
Εκτύπωση των k στοιχείων (**maxHeap.printSuspects()**): Η αφαίρεση και εκτύπωση των k πιο ύποπτων καταθετών από τον δυαδικό σωρό έχει πολυπλοκότητα O(log k) για κάθε αφαίρεση, με συνολική πολυπλοκότητα **O(k log k)** για την αφαίρεση και εκτύπωση όλων των k στοιχείων.

Συνδυάζοντας τα παραπάνω, η συνολική πολυπλοκότητα της printTopLargeDepositors είναι κυρίως επηρεασμένη από τη γραμμική διάσχιση του δέντρου (O(N)) και την εκτύπωση των στοιχείων (O(k log k)). Αν το k είναι σχετικά μικρό σε σχέση με το N, τότε η πολυπλοκότητα τείνει προς **O(N)**. Αν το k είναι συγκρίσιμο με το N, τότε η πολυπλοκότητα επηρεάζεται και από την εκτύπωση των στοιχείων, οδηγώντας σε μια πιθανή συνολική πολυπλοκότητα **O(N + k log k)**.

- **void printByAFM()**: Τυπώνει τα στοιχεία όλων των κόμβων του δέντρου, ταξινομημένα σε αύξουσα σειρά ως προς τον ΑΦΜ. Χρησιμοποιεί την βοηθητική μέθοδο **printInOrder(root)**, η οποία αναδρομικά επισκέπτεται κάθε κόμβο του δέντρου σε ενδοδιατεταγμένη (in-order) σειρά και εκτυπώνει τα στοιχεία τους. Αυτός ο τρόπος διάσχισης εγγυάται ότι τα στοιχεία των κόμβων θα εκτυπωθούν σε αύξουσα σειρά των ΑΦΜ.
  Η μέθοδος επισκέπτεται κάθε κόμβο ακριβώς μία φορά. Επομένως, η πολυπλοκότητα της διάσχισης είναι O(N), όπου N είναι ο συνολικός αριθμός των κόμβων στο δέντρο.

Μέρος Γ

Στην μέθοδο main της κλάσης RandomizedBST, κατασκευάσαμε ένα απλό μενού διαχείρισης με το οποίο μπορεί κάποιος να εκτελεί όλες τις λειτουργίες της δομής.